

# Ch 06: Interrupts and Device Drivers

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Fall 2026



**EAST TEXAS A&M**

1 Interrupts and Device Drivers

2 Console Input and Output

3 Concurrency and Timer Interrupts



**EAST TEXAS A&M**

# Ch 06: Interrupts and Device Drivers

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Fall 2026



**EAST TEXAS A&M**

- A device **driver** is the code in an operating system that manages a particular device
  - configures the device hardware
  - tells the device to perform operations
  - handles resulting interrupts when operations complete
  - interacts with processes using the device



# Interrupts

- Devices that need attention from the operating system can be configured to generate interrupts
  - one type of trap (along with system calls and faults)
- Kernel trap handler recognizes when a device has raised an interrupt
  - calls the driver's interrupt handler
  - in xv6 this happens in `devintr()`



# Device Driver General Layout

Common device drivers code has two contexts

- **top half** runs in a process's kernel thread
  - called via system calls
    - such as `read()` and `write()` to a fd opened on a special device
  - initiates device operation usually
    - e.g. perform read or write I/O
- **bottom half** executes when device interrupt occurs
  - eventually the device completes the operation and raises an interrupt
  - driver's interrupt handler, acting as bottom half, figures out which operation completed
  - wakes up waiting process if appropriate
  - tells hardware to start work on the next operation



# Summary

- Device driver is code in an OS that manages a particular device
- Interrupts are used by devices to notify the OS they need attention
- Device drivers commonly have two contexts
  - **top half** runs in process's kernel thread, is invoked by a system call, initiates device operation, may cause the calling process to block
  - **bottom half** execute when device raises interrupt, completes the operation, wakes up process if appropriate, tells device to start work on the next operation



# Ch 06: Console Input and Output

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Fall 2026



**EAST TEXAS A&M**

# Console Input (1 of 2)

- Console driver (`kernel/console.c`) serves as simple illustration of driver structure
  - UART serial-port hardware attached to the RISC-V
  - console driver accumulates line of input
  - user processes (such as the shell) use `read()` system call to fetch lines
- UART hardware is a 16550 serial link chip (emulated by QEMU)
  - UART hardware appears as set of **memory-mapped** control registers
    - e.g. some physical addresses are connected to the UART device so that loads or stores to those addresses interact with the device hardware rather than RAM
  - memory-mapped addresses for UART start at `0x10000000`, or UART0 (see `kernel/memorylayout.h` and `kernel/uart.c`)
  - `main()` calls `consoleinit()` (in `kernel/console.c`) which calls `uartinit()` (in `kernel/uart.c`) to configure UART
    - generate **receive interrupt** when UART receives each byte of input
    - **transmit complete interrupt** each time UART finishes sending a byte of output



## Console Input (2 of 2)

- xv6 shell reads from the console by way of a file description opened by `init.c` (`open(console, "O_RDWR")`)
  - calls to `read()` make way through kernel to `consoleread()`
    - waits for input to arrive (via interrupts)
    - copies the input to user space
    - after whole line has arrived, returns to the user process
  - when user types a character, UART raises an interrupt
    - activates xv6 trap handler
    - trap handler calls `devintr()`
    - Queries PLIC to tell which device interrupted, if it was UART `devintr()` calls `uartintr()`
    - `uartintr()` reads waiting input character from UART and hands to `consoleintr()`



# Console Output

- `write()` system call on fd connected to console eventually arrives at `uartputc()`
- Device driver maintains an output buffer `uart_tx_buf`
  - `uartputc()` appends each character to the buffer
  - calls `uartstart()` to start device transmitting
  - returns
- When UART finishes sending a byte, it generates an interrupt
  - `uartintr()` calls `uartstart()` which checks that the device really has finished sending
  - hands device next buffered output character



# General Device Driver Design Pattern

- Device activity is decoupled from process activity via buffering and interrupts
  - Console driver can process input even when no process is waiting to read it
    - subsequent read will see the input (taken from buffer)
  - Similarly processes can send output without having to wait for the device
- This decoupling can increase performance by allowing processes to execute concurrently with device I/O and is particularly important when the device is slow
  - called **I/O Concurrency**



# Summary

- Console driver accepts characters typed by a human
  - user processes, such as the shell, use the `read()` system call to fetch lines of input from the console
- UART hardware appears to software as a set of **memory-mapped** control registers
  - Some devices (but not all) may use this type of memory mapped control
  - Others might use direct-memory-access (DMA) methods
- A `write()` system call on a file descriptor connected to the console eventually arrives at `uartputc()`
  - Each time UART finishes sending a byte, it generates an interrupt
- A general pattern is the decoupling of device activity via buffering and interrupts
- This decoupling increase performance by allowing processes to execute concurrently with I/O devices
  - **I/O concurrency**



# Ch 06: Concurrency and Timer Interrupts

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science  
East Texas A&M University

Fall 2026



**EAST TEXAS A&M**

# Concurrency in Drivers

- Note calls to `acquire()` in `consoleread()` and `consoleintr()`
  - These acquire a mutual exclusion **lock**
  - protects console driver's data structures from concurrent access
- Three concurrency dangers
  - ① two processes on different CPUs might call `consoleread()` at the same time
  - ② the hardware might ask a CPU to deliver a console interrupt while that CPU is already inside `consoleread()`
  - ③ the hardware might deliver a console interrupt on a different CPU while `consoleread()` is executing
- Interrupt handlers typically do relatively little work (e.g. just copy the input data to a buffer), and wake up top-half code to do the rest.

Next unit/chapter will discuss concurrency locks in more detail, but for now think of the `acquire()` and `release()` as defining a **critical section**: only 1 cpu/process/thread will ever be executing code in a critical section.



# Timer Interrupts

- xv6 uses **timer interrupts**
  - to maintain its idea of the current time and
  - to switch among compute-bound processes
- Timer interrupts come from clock hardware attached to each RISC-V CPU
  - xv6 programs each CPU's clock hardware to interrupt the CPU periodically
- `start.c` sets control bits to allow access to timer control registers and asks for first timer interrupt
  - `time` control register contains a count that the hardware increments at a steady rate
  - `stimecmp` register contains a time at which the CPU will raise a timer interrupt
    - setting `stimecmp` to the current value of `time` plus `x` will schedule an interrupt `x` time units in the future
    - for qemu's RISC-V emulation, 1000000 time units is roughly a tenth of a second
  - Timer interrupts arrive via `usertrap()` or `kerneltrap()` and `devintr()`, like other device interrupts
    - arrive with `scause` low bits set to five,
    - `devintr()` in `trap.c` detects this situation and calls `clockintr()`
    - `clockintr()` wakes up any processes waiting in the `pause` system call, and schedules the next timer interrupt by writing `stimecmp`



EAST TEXAS A&M

- xv6, like many operating systems, allows interrupts and even context switches (via `yield()`) while executing in the kernel
  - the reason is to retain quick response times during complex system calls that run for a long time
  - but allowing interrupts in the kernel is the source of some complexity
- Supporting all the devices on a typical computer is much work
  - many devices
  - devices have many features
  - in many OS, the drivers account for more code than the core kernel
- The UART driver uses **programmed I/O** since software is driving the data movement
  - programmed I/O is simple, but too slow to be used at high data rates
  - **direct memory access** (DMA) devices allow hardware to directly write incoming data to RAM and read outgoing data from RAM
- The console appears to applications as a regular file, and applications read and write output using the `read()` and `write()` system calls.



# Summary

- In order for device drivers to work correctly, must handle concurrency correctly
  - We will discuss this more in the next unit
- Device driver bottom-half interrupt handlers typically do relatively little work just copy the input data to a buffer, then wake up the top-half code.
- Timer interrupts are used by an OS to switch among compute-bound processes (multiprogramming)
  - In xv6, timer interrupts are handled in `usertrap()` and `kerneltrap()` (which call `devintr()`)
  - Forces a context switch via `yield`, allows other processes to be time-shared
- In Unix-like system, special device files appear as regular files, and are one way to hook in device driver code for use by user programs to communicate with devices.
- The example UART driver used **programmed I/O**, directly controlled by the OS kernel software
  - For high data rate devices, probably need to use **DMA** (direct memory access) and maybe reduce interrupts or use **polling**.



# References



**EAST TEXAS A&M**